

```
import string

# Символи пунктуації
punct = set(string.punctuation)

# Морфологічні правила, що використовуються для класифікації невідомих слів
noun_suffix = ["action", "age", "ance", "cy", "dom", "ee", "ence", "er", "hood", "ion", "ism", "ist", "ity", "ling", "mer"]
verb_suffix = ["ate", "ify", "ise", "ize"]
adj_suffix = ["able", "ese", "ful", "i", "ian", "ible", "ic", "ish", "ive", "less", "ly", "ous"]
adv_suffix = ["ward", "wards", "wise"]
```

```
def assign_unk(tok):
    """
    Призначення міток для невідомих слів
    """
    # Цифри
    if any(char.isdigit() for char in tok):
        return "--unk_digit--"

    # Пунктуація
    elif any(char in punct for char in tok):
        return "--unk_punct--"

    # Великі літери
    elif any(char.isupper() for char in tok):
        return "--unk_upper--"

    # Іменники
    elif any(tok.endswith(suffix) for suffix in noun_suffix):
        return "--unk_noun--"

    # Дієслова
    elif any(tok.endswith(suffix) for suffix in verb_suffix):
        return "--unk_verb--"

    # Прикметники
    elif any(tok.endswith(suffix) for suffix in adj_suffix):
```

```

        return "--unk_adj--"

# Прислівники
elif any(tok.endswith(suffix) for suffix in adv_suffix):
    return "--unk_adv--"

return "--unk--"

```

```

import re

text = """
The suspected shooter has been identified as an Afghan national who entered the United States
in 2021 and at some point lived in Washington state, according to multiple people familiar with
the investigation who spoke on the condition of anonymity to discuss sensitive information.
Two of those people said the suspect was Rahmanullah Lakanwal
"""

# Tokenize the text, separating words and punctuation. This regex matches either a sequence of word characters (including
# or any non-whitespace, non-word character (i.e., punctuation).
tokens = re.findall(r"[\w']+|[\^\s\w]", text)

# Process each token and apply assign_unk
results = []
for token in tokens:
    # Only process non-empty tokens after stripping potential whitespace
    if token.strip():
        label = assign_unk(token)
        results.append(f'"{token}" - "{label}"')

# Print the results
for res in results:
    print(res)

```

```

"The" - "--unk_upper--"
"suspected" - "--unk--"
"shooter" - "--unk_noun--"
"has" - "--unk--"
"been" - "--unk--"
"identified" - "--unk--"
"as" - "--unk--"

```

```
"an" - "--unk--"
"Afghan" - "--unk_upper--"
"national" - "--unk--"
"who" - "--unk--"
"entered" - "--unk--"
"the" - "--unk--"
"United" - "--unk_upper--"
"States" - "--unk_upper--"
"in" - "--unk--"
"2021" - "--unk_digit--"
"and" - "--unk--"
"at" - "--unk--"
"some" - "--unk--"
"point" - "--unk--"
"lived" - "--unk--"
"in" - "--unk--"
"Washington" - "--unk_upper--"
"state" - "--unk_verb--"
"," - "--unk_punct--"
"according" - "--unk--"
"to" - "--unk--"
"multiple" - "--unk--"
"people" - "--unk--"
"familiar" - "--unk--"
"with" - "--unk--"
"the" - "--unk--"
"investigation" - "--unk_noun--"
"who" - "--unk--"
"spoke" - "--unk--"
"on" - "--unk--"
"the" - "--unk--"
"condition" - "--unk_noun--"
"of" - "--unk--"
"anonymity" - "--unk_noun--"
"to" - "--unk--"
"discuss" - "--unk--"
"sensitive" - "--unk_adj--"
"information" - "--unk_noun--"
"." - "--unk_punct--"
"Two" - "--unk_upper--"
"of" - "--unk--"
"those" - "--unk--"
"people" - "--unk--"
"said" - "--unk--"
"the" - "--unk--"
```

```
"suspect" - "--unk--"  
"was" - "--unk--"  
"Rahmanullah" - "--unk_upper--"  
"Lakanwal" - "--unk_upper--"
```

```
import nltk  
import nltk.corpus  
from nltk.corpus import brown  
  
nltk.download("brown")
```

```
[nltk_data] Downloading package brown to /root/nltk_data...  
[nltk_data]   Unzipping corpora/brown.zip.  
True
```

```
#nltk.download()
```

```
import nltk  
import random  
from collections import Counter  
  
# Завантаження корпусу (приклад для Brown)  
nltk.download('brown')  
from nltk.corpus import brown  
  
# Отримання тегованих речень  
tagged_sents = list(brown.tagged_sents())  
  
# Перемішування та розділення на навчальну і тестову вибірки  
random.seed(42)  
random.shuffle(tagged_sents)  
  
split_point = int(len(tagged_sents) * 0.8)  
train_sents = tagged_sents[:split_point]  
test_sents = tagged_sents[split_point:]  
  
# Запис у файли  
def write_tagged_sentences(sentences, filename):  
    with open(filename, 'w', encoding='utf-8') as f:  
        for sentence in sentences:
```

```
        for word, tag in sentence:
            f.write(f"{word}\t{tag}\n")
        f.write("\n") # Порожній рядок між реченнями

write_tagged_sentences(train_sents, "brown_training.pos")
write_tagged_sentences(test_sents, "brown_test.pos")

# Створення словника - виправлена версія
word_counter = Counter()
for sentence in train_sents:
    for word, _ in sentence:
        word_counter[word] += 1

# Збереження слів, які з'являються принаймні двічі
vocab = {word for word, count in word_counter.items() if count >= 2}

with open("brown_vocab.txt", 'w', encoding='utf-8') as f:
    for word in sorted(vocab):
        f.write(f"{word}\n")

# Створення файлу тестових слів
with open("brown_test_words.txt", 'w', encoding='utf-8') as f:
    for sentence in test_sents:
        for word, _ in sentence:
            f.write(f"{word}\n")
        f.write("\n") # Порожній рядок між реченнями
```

```
[nltk_data] Downloading package brown to /root/nltk_data...
[nltk_data]   Package brown is already up-to-date!
```

```
# Виправлення 1: Додавання спеціальних токенів до словника
def preprocess(vocab, data_fp):
    """
    Попередня обробка даних
    """
    orig = []
    prep = []

    # Додаю спеціальний токен для кінця речення
    if "--n--" not in vocab:
        vocab["--n--"] = len(vocab)
```

```
# Читання даних
with open(data_fp, "r") as data_file:
    for cnt, word in enumerate(data_file):
        # Кінець речення
        if not word.split():
            orig.append(word.strip())
            word = "--n--"
            prep.append(word)
            continue

        # Обробка невідомих слів
        elif word.strip() not in vocab:
            orig.append(word.strip())
            word = assign_unk(word.strip()) # Виправлення: додано strip()
            prep.append(word)
            continue

        else:
            orig.append(word.strip())
            prep.append(word.strip())

assert(len(orig) == len(open(data_fp, "r").readlines()))
assert(len(prepare) == len(open(data_fp, "r").readlines()))

return orig, prep
```

```
import numpy as np
import pandas as pd
from collections import defaultdict
import math

# Завантаження навчального корпусу
with open("brown_training.pos", 'r') as f:
    training_corpus = f.readlines()

# Завантаження словника
with open("brown_vocab.txt", 'r') as f:
    voc_l = f.read().split('\n')
```

```
# Створення словника з індексами для слів
vocab = {}
for i, word in enumerate(sorted(voc_l)):
    vocab[word] = i

# Завантаження тестового корпусу
with open("/content/brown_test.pos", 'r') as f:
    y = f.readlines()

# Попередня обробка тестових слів
_, prep = preprocess(vocab, "/content/brown_test_words.txt")
```

prep

```

ever',
'seeing',
'teeth',
'that',
'were',
'quite',
'so',
'white',
'and',
'at',
'the',
'same',
'time',
'quite',
'so',
'--unk_adj--',
'not',
'--unk--',
'.',
'--n--',
'.',
'--n--',
'We',
'are',
'saying',
'that',
'the',
'pain',
'was',
'bad',
...]
```

```

# Виправлення 2: Виправлена функція get_word_tag
def get_word_tag(line, vocab):
    """
    Отримання слова та його тегу з рядка корпусу.
    """
    if not line.split():
        word = "--n--"
        tag = "--s--"
        return word, tag # Виправлення: додано повернення значення
    else:
        parts = line.split()
        if len(parts) >= 2:
```



```
word, tag = parts[0], parts[1]
if word not in vocab:
    # Обробка невідомих слів
    word = assign_unk(word)
return word, tag
else:
    # Обробка некоректних рядків
    return "--n--", "--s--" # Виправлення: додано повернення для некоректних рядків
```

```
def create_dictionaries(training_corpus, vocab):
    """
    Створення словників частот.
    """
    emission_counts = defaultdict(int)
    transition_counts = defaultdict(int)
    tag_counts = defaultdict(int)

    # Початковий тег
    prev_tag = '--s--'

    for word_tag in training_corpus:
        word, tag = get_word_tag(word_tag, vocab)

        # Збільшення лічильника переходів
        transition_counts[(prev_tag, tag)] += 1

        # Збільшення лічильника емісій
        emission_counts[(tag, word)] += 1

        # Збільшення лічильника тегів
        tag_counts[tag] += 1

        # Оновлення попереднього тегу
        prev_tag = tag

    return emission_counts, transition_counts, tag_counts
```

```
emission_counts, transition_counts, tag_counts = create_dictionaries(training_corpus, vocab)
```

```
emission_counts
```

```
(UNK, <padding>) = 00
```

```
( 'NN', 'meeting' ): 99,
( 'JJ', 'long' ): 425,
( 'RB', 'ago' ): 194,
( 'VBD', 'swore' ): 12,
( 'QL', 'All' ): 49,
( 'RB', 'right' ): 61,
( 'JJ', 'enthusiastic' ): 24,
( 'IN', 'over' ): 644,
( 'RB', 'newly' ): 22,
( 'VBN', 'acquired' ): 15,
( 'NP', 'Claude' ): 10,
( 'VBD', '--unk--' ): 386,
( 'NN', 'pleasure' ): 48,
( 'NP', 'Poussin' ): 2,
( 'NN', 'exhibit' ): 8,
( 'JJ', 'able' ): 181,
( 'NP', 'Paris' ): 52,
( 'VB', 'keep' ): 200,
( 'NN', 'bloodstream' ): 4,
( 'JJ', 'clean' ): 42,
( 'AP', 'former' ): 100,
...})
```

```
def create_transition_matrix(alpha, tag_counts, transition_counts):
```

```
    """
```

```
    Створення матриці переходів A.
```

```
    """
```

```
    all_tags = sorted(tag_counts.keys())
```

```
    num_tags = len(all_tags)
```

```
    A = np.zeros((num_tags, num_tags))
```

```
    for i in range(num_tags):
```

```
        for j in range(num_tags):
```

```
            key = (all_tags[i], all_tags[j])
```

```
            count = 0
```

```
            if key in transition_counts:
```

```
                count = transition_counts[key]
```

```
            count_prev_tag = tag_counts[all_tags[i]]
```

```
            # Застосування згладжування
```

```
A[i, j] = (count + alpha) / (count_prev_tag + alpha * num_tags)

return A
```

```
A = create_transition_matrix(0.0001, tag_counts, transition_counts)
```

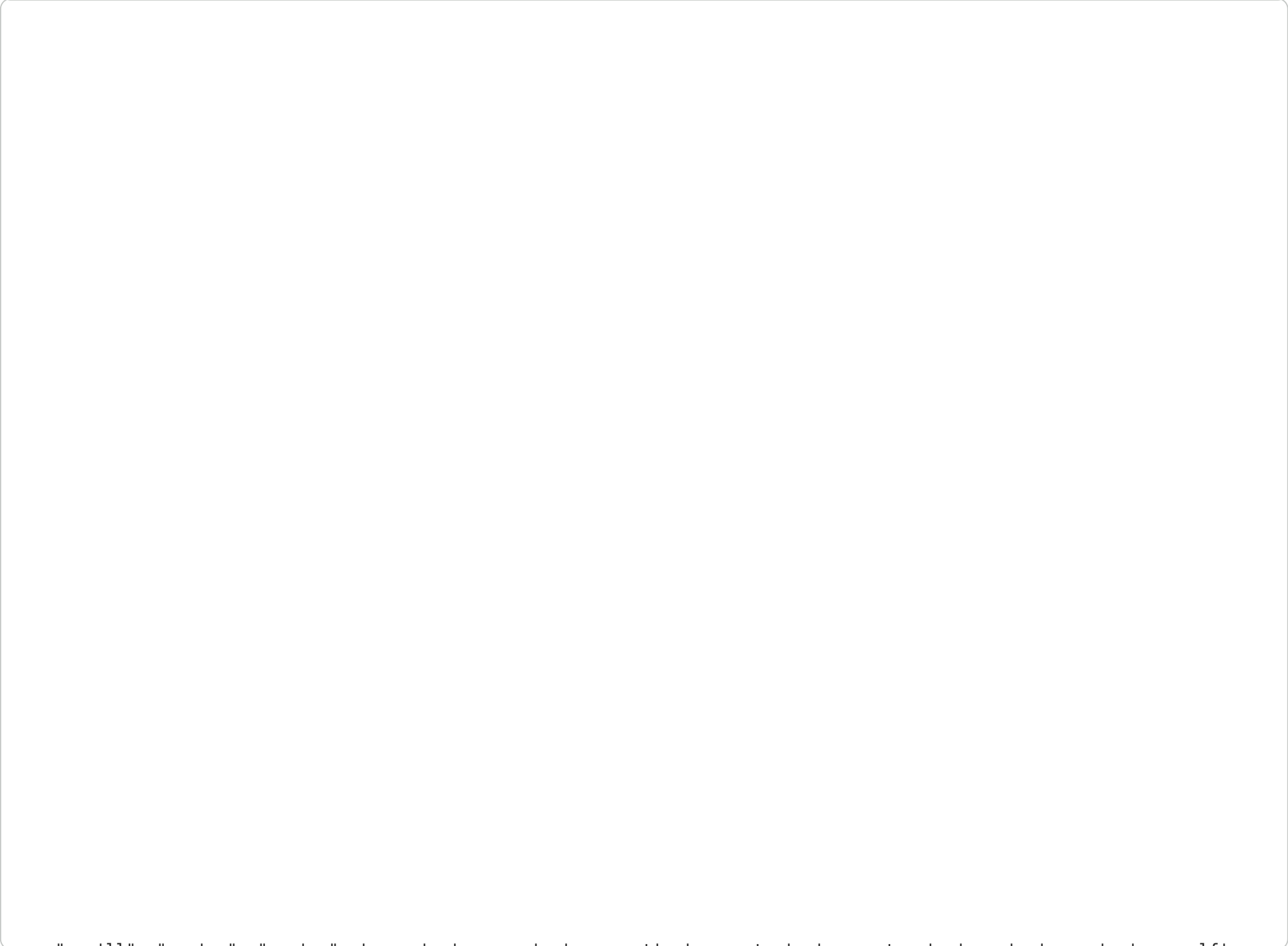
A

```
array([[3.86033166e-07, 1.08089672e-01, 3.86033166e-07, ...,
        1.15813810e-02, 3.86033166e-07, 3.86071769e-03],
       [1.43883958e-08, 1.43883958e-08, 6.61867647e-03, ...,
        1.43883958e-08, 1.43883958e-08, 4.31666264e-04],
       [5.56215192e-04, 5.56215192e-04, 6.11781095e-03, ...,
        5.56159576e-08, 5.56159576e-08, 1.72410025e-02],
       ...,
       [4.25842075e-01, 1.41942627e-05, 1.41942627e-05, ...,
        1.41942627e-05, 1.41942627e-05, 1.41942627e-05],
       [1.41942627e-05, 1.41942627e-05, 1.41942627e-05, ...,
        1.41942627e-05, 1.41942627e-05, 1.41942627e-05],
       [2.87038900e-04, 1.43512274e-08, 2.87038900e-04, ...,
        1.43526626e-04, 8.61087997e-04, 1.43512274e-08]])
```

```
all_tags = sorted(tag_counts.keys())
all_tags[0]
```

```
...
```

```
vocab.keys()
```



```
you ll , you re , you ve , young , younger , youngest , youngster , youngsters , your , yours , yourself ,
'yourselves', 'youth', 'youthful', 'youths', 'yow', 'yrs.', 'zeal', 'zealous', 'zealously', 'zero', 'zero-magnitude',
'zest', 'zigzagging', 'zinc', 'zing', 'zodiacal', 'zone', 'zones', 'zoning', 'zoo', '--n--'1)
```

```
def create_emission_matrix(alpha, tag_counts, emission_counts, vocab):
    """
    Створення матриці емісій B.
    """
    all_tags = sorted(tag_counts.keys())
    num_tags = len(tag_counts)
    num_words = len(vocab)

    B = np.zeros((num_tags, num_words))

    for i in range(num_tags):
        for j in range(num_words):
            #print(all_tags[i], vocab[j])
            key = (all_tags[i], vocab[j])

            count = 0
            if key in emission_counts:
                count = emission_counts[key]

            count_tag = tag_counts[all_tags[i]]

            # Застосування згладжування
            B[i, j] = (count + alpha) / (count_tag + alpha * num_words)

    return B
```

```
B = create_emission_matrix(0.0001, tag_counts, emission_counts, sorted(vocab.keys()))
```

```
B.shape
```

```
(451, 27049)
```

```
# Виправлення 3: Оновлення функцій Вітербі для роботи з невідомими словами
def initialize(states, tag_counts, A, B, corpus, vocab):
    """
```

```
    Ініціалізація алгоритму Вітербі
```

```
"""
Ініціалізація алгоритму Вітербі.
"""
```

```
num_tags = len(tag_counts)
```

```
# Ініціалізація матриць best_probs та best_paths
```

```
best_probs = np.zeros((num_tags, len(corpus)))
```

```
best_paths = np.zeros((num_tags, len(corpus)), dtype=int)
```

```
# Індекс початкового стану
```

```
s_idx = states.index("--s--")
```

```
# Заповнення першого стовпця best_probs
```

```
word = corpus[0]
```

```
word_idx = vocab.get(word, vocab.get("--unk--", 0)) # Виправлення: обробка невідомих слів
```

```
for i in range(num_tags):
```

```
    # A та B вже в логарифмічному просторі
```

```
    best_probs[i, 0] = A[s_idx, i] + B[i, word_idx]
```

```
return best_probs, best_paths
```

```
def viterbi_forward(A, B, corpus, best_probs, best_paths, vocab):
```

```
    """
```

```
    Пряме проходження алгоритму Вітербі.
    """
```

```
    num_tags = best_probs.shape[0]
```

```
    for i in range(1, len(corpus)):
```

```
        word = corpus[i]
```

```
        word_idx = vocab.get(word, vocab.get("--unk--", 0)) # Виправлення: обробка невідомих слів
```

```
        # Calculate emission scores for the current word for all tags
```

```
        emission_scores_for_word = B[:, word_idx] # Shape (num_tags,)
```

```
        # Calculate scores from previous best probabilities plus transition probabilities
```

```
        # best_probs[:, i-1] has shape (num_tags,)
```

```
        # A has shape (num_tags, num_tags)
```

```
        # By broadcasting, best_probs[:, i-1][:, None] + A results in (num_tags, num_tags) matrix
```

```
        # where element (k, j) is best_probs[k, i-1] + A[k, j]
```

```
        scores_from_prev_tags = best_probs[:, i-1][:, np.newaxis] + A # Shape (num_tags, num_tags)
```

```
        # For each current tag i, find the maximum score from any previous tag k
```

```

# For each current tag j, find the maximum score from any previous tag k
# max_scores_to_current_tags will be a vector of shape (num_tags,)
# max_scores_to_current_tags[j] = max_k (best_probs[k, i-1] + A[k, j])
best_probs[:, i] = np.max(scores_from_prev_tags, axis=0) + emission_scores_for_word

# To update best_paths, we need the indices of the maximums.
# This requires argmax along the k dimension for each j
best_paths[:, i] = np.argmax(scores_from_prev_tags, axis=0)

return best_probs, best_paths

```

```

def viterbi_backward(best_probs, best_paths, corpus, states):
    """
    Зворотне проходження алгоритму Вітербі.
    """
    m = best_paths.shape[1]
    z = [None] * m
    pred = [None] * m

    # Знаходження найкращого тегу для останнього слова
    best_prob_for_last_word = float('-inf')

    for k in range(best_probs.shape[0]):
        if best_probs[k, m-1] > best_prob_for_last_word:
            best_prob_for_last_word = best_probs[k, m-1]
            z[m-1] = k

    pred[m-1] = states[z[m-1]]

    # Зворотне проходження для знаходження найкращих тегів
    for i in range(m-2, -1, -1):
        z[i] = best_paths[z[i+1], i+1]
        pred[i] = states[z[i]]

    return pred

```

```

def compute_accuracy(pred, y):
    """
    Обчислення точності моделі POS-тегування.
    """

```



```
"""
num_correct = 0
total = 0

for prediction, y_item in zip(pred, y):
    y_item = y_item.strip()
    word_tag_tuple = y_item.split('\t')

    if len(word_tag_tuple) != 2:
        continue

    word, tag = word_tag_tuple

    if tag == prediction:
        num_correct += 1

    total += 1

return num_correct / total
```

```
# Створення словників
emission_counts, transition_counts, tag_counts = create_dictionaries(training_corpus, vocab)

# Отримання списку всіх тегів
states = sorted(tag_counts.keys())

# Параметр згладжування
alpha = 0.001

# Створення індексованого словника слів
word_to_index = {}
for i, word in enumerate(sorted(vocab.keys())):
    word_to_index[word] = i

# Додавання спеціальних токенів
for special_token in ["--n--", "--unk--", "--unk_digit--", "--unk_punct--", "--unk_upper--",
                     "--unk_noun--", "--unk_verb--", "--unk_adj--", "--unk_adv--"]:
    if special_token not in word_to_index:
        word_to_index[special_token] = len(word_to_index)
```

```
# Створення матриць переходів та емісій (у логарифмічному просторі)
A = np.log(create_transition_matrix(alpha, tag_counts, transition_counts))
B = np.log(create_emission_matrix(alpha, tag_counts, emission_counts, list(word_to_index.keys())))
```

```
# Ініціалізація алгоритму Вітербі
best_probs, best_paths = initialize(states, tag_counts, A, B, prep, word_to_index)
```

```
# Пряме проходження
best_probs, best_paths = viterbi_forward(A, B, prep, best_probs, best_paths, word_to_index)
```

```
# Зворотне проходження для отримання найкращої послідовності тегів
pred = viterbi_backward(best_probs, best_paths, prep, states)
```

```
# Обчислення точності
accuracy = compute_accuracy(pred, y)
print(f"Точність моделі: {accuracy:.4f}")
```

Точність моделі: 0.9459

```
import nltk
from nltk import word_tokenize
from nltk.corpus import brown

# Завантаження необхідних ресурсів
nltk.download('punkt')
nltk.download('averaged_perceptron_tagger')
nltk.download('brown')
nltk.download('punkt_tab')
nltk.download('averaged_perceptron_tagger_eng')
```

```
# Приклад тексту
text = """The suspected shooter has been identified as an Afghan national who entered the United States in 2021 and at sc
```

```
# Токенізація тексту
tokens = word_tokenize(text)

# POS-тегування з використанням NLTK
nltk_tags = nltk.pos_tag(tokens)

print("Результат POS-тегування NLTK:")
for word, tag in nltk_tags:
    print(f"{word}: {tag}")

import random

# Оцінка точності NLTK POS-tagger на корпусі

# Отримання тегованих речень
tagged_sents = list(brown.tagged_sents())

# Перемішування та розділення на навчальну і тестову вибірки
random.seed(42)
random.shuffle(tagged_sents)

split_point = int(len(tagged_sents) * 0.8)
train_data = tagged_sents[:split_point]
test_data = tagged_sents[split_point:]

# Навчання NLTK тегера
from nltk.tag import UnigramTagger, BigramTagger
unigram_tagger = UnigramTagger(train_data)
bigram_tagger = BigramTagger(train_data, backoff=unigram_tagger)

# Оцінка точності
nltk_accuracy = bigram_tagger.accuracy(test_data)
print(f"Точність NLTK BigramTagger: {nltk_accuracy:.4f}")

# Порівняння з нашою реалізацією
print(f"Точність нашої реалізації: {accuracy:.4f}")
print(f"Різниця: {abs(accuracy - nltk_accuracy):.4f}")
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
```

```
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data] /root/nltk_data...
[nltk_data] Unzipping taggers/averaged_perceptron_tagger.zip.
[nltk_data] Downloading package brown to /root/nltk_data...
[nltk_data] Package brown is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt_tab.zip.
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data] /root/nltk_data...
[nltk_data] Unzipping taggers/averaged_perceptron_tagger_eng.zip.
```

Результат POS-тегування NLTK:

The: DT
suspected: JJ
shooter: NN
has: VBZ
been: VBN
identified: VBN
as: IN
an: DT
Afghan: NNP
national: JJ
who: WP
entered: VBD
the: DT
United: NNP
States: NNPS
in: IN
2021: CD
and: CC
at: IN
some: DT
point: NN
lived: VBD
in: IN
Washington: NNP
state: NN
,: ,
according: VBG
to: TO
multiple: JJ
people: NNS
familiar: JJ
with: IN
the: DT
investigation: NN

who: WP
spoke: VBD
on: IN
the: DT
condition: NN
of: IN
anonymity: NN
to: TO
discuss: VB
sensitive: JJ

Start coding or [generate](#) with AI.